

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25 100
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	VARSHA VARDHINI. S	Reg.No.:	192511488
Department:	BE - CSE	Date:	

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

85% 18

- Smartphone Performance Optimization**
A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.
- Cloud Server Memory Bottleneck**
A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.
- Autonomous Vehicle Data Processing**
An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.
- AI Inference Edge Device**
An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.
- High-Performance Gaming Console**
A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Code of Conduct

I Vaishya Vaadhini S / 192511488 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S. Vaishya

MARKS ALLOCATION

	Understanding of Industry Problem (20%)	Application of Engineering Concepts (25%)	Solution Design (25%)	Use of Tools (15%)	Analysis (10%)	
Q1	1	1.25	1	0.75	0.5	4.5
Q2	1	1.25	1	0.75	0.5	4.5
Q3	0.5	1	1.25	0.75	0.25	3.75
Q4	0.5	1	1.25	0.5	0.75	4
Q5	1	1.25	1.25	0.5	0.5	4.5

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints(1)	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem(1.25)	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards(1.25)	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data(0.75)	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation(0.5)	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

21.25

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL2- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	VARSHA VARDHINI.S	Reg.No.:	192511488
Department:	BE CSE	Date:	

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization

A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

ANSWER :

Smartphone Performance Optimization

The lag during app switching mainly occurs because the processor has to retrieve application instructions and data from different levels of memory. When the required data is already available in the cache, the CPU can access it quickly. However, when a cache miss occurs, the processor must access slower levels of memory, especially DRAM. If this happens frequently while switching between multiple applications, the user experiences delays and reduced responsiveness.

Role of L1, L2, L3 Cache and DRAM

L1 Cache:

L1 is the smallest and fastest cache, located closest to each CPU core. It stores the most frequently used instructions and data of the currently running application. A high L1 cache hit rate helps the processor execute tasks with minimum delay.

L2 Cache:

L2 is larger than L1 but slightly slower. It stores recently used data that cannot be accommodated in L1. During app switching, L2 can reduce the need to access DRAM if the required application data is still available.

L3 Cache:

L3 is larger and generally shared among multiple CPU cores. This is particularly useful in a smartphone because different applications and background processes may run on different cores. A shared L3 cache allows recently accessed data to remain available even when the workload moves between cores.

DRAM:

DRAM acts as the main memory and stores active applications, operating system data and background processes. It has much higher capacity than cache but also higher latency. Therefore, frequent DRAM accesses can become a major reason for lag.

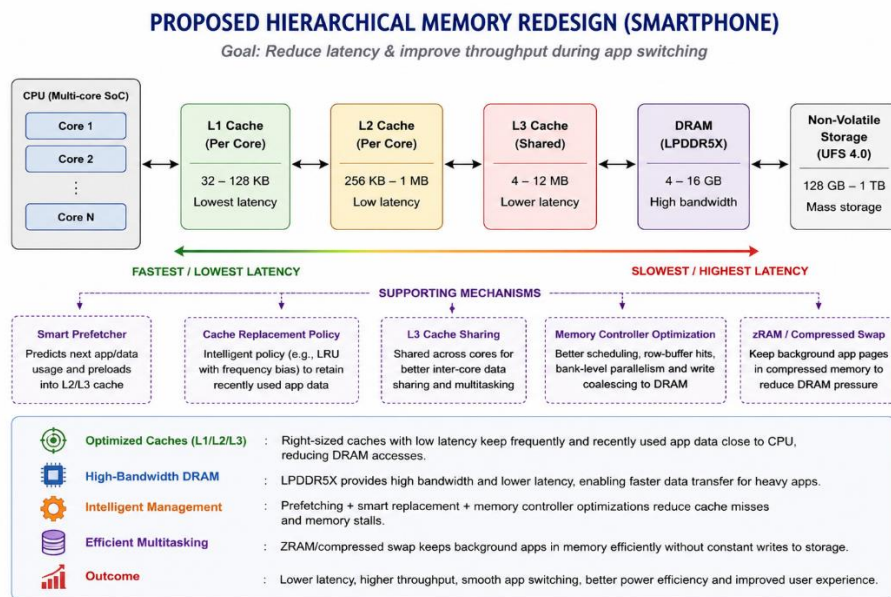
Proposed Hierarchical Memory Redesign

A suitable memory hierarchy is:

CPU Core → L1 Cache → Private L2 Cache → Shared L3 Cache → High-Bandwidth LPDDR DRAM → Flash Storage

The redesigned system should use:

- **Optimized L1 cache** for the most frequently accessed instructions and data of the active application.
- **Larger private L2 caches** to retain recently used application data.
- **A larger shared L3 cache** to improve data sharing between CPU cores and reduce cache misses during app switching.
- **Prefetching mechanisms** to load likely required application data into the cache before it is requested.
- **High-bandwidth LPDDR DRAM** to reduce the delay when data must be fetched beyond the cache hierarchy.
- Intelligent cache management can prioritize the foreground application while retaining useful data from recently used applications.



Performance Analysis

In the existing system, frequent cache misses force the processor to access DRAM, increasing memory access latency. The proposed hierarchy reduces this problem by keeping frequently and recently used data closer to the processor.

For example, when a user switches from one recently used application to another, part of its working data may still be available in the L3 or L2 cache instead of being fetched again from DRAM. This reduces memory access time and allows the CPU to continue execution with fewer stalls.

Therefore, the redesigned hierarchy can provide:

- Reduced application switching latency
- Higher cache hit rate
- Fewer DRAM accesses
- Better data sharing between CPU cores
- Improved processor throughput
- Smoother multitasking and better user experience

Conclusion

The main solution is not simply increasing DRAM capacity. A balanced hierarchy using fast L1 cache, larger L2 cache, shared L3 cache and high-bandwidth DRAM is more effective. By keeping frequently and recently used application data closer to the CPU, the smartphone can reduce memory latency and improve throughput during multitasking and app switching.

2. Cloud Server Memory Bottleneck

A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

ANSWER :

Cloud Server Memory Bottleneck

A cloud database server handles a large number of queries simultaneously. Frequently accessed data such as database indexes, query instructions and active records should be available as close to the CPU as possible. If the data is not found in the cache, it must be fetched from DRAM. When physical memory becomes insufficient, the operating system may move memory pages between DRAM and secondary storage. This paging process can significantly increase database query latency.

Cache Hierarchy vs Virtual Memory Paging

Cache Hierarchy	Virtual Memory Paging
Uses L1, L2 and L3 caches	Uses DRAM and secondary storage
Keeps frequently used data close to the CPU	Moves memory pages when physical memory is insufficient
Very low access latency	Much higher latency during page faults
Small capacity but high speed	Large virtual capacity but slower access
Improves CPU performance and throughput	Prevents memory exhaustion but can reduce performance if excessive

The cache hierarchy is mainly used to **speed up memory access**, whereas virtual memory is used to **extend the available memory space**. Therefore, virtual memory should not be treated as a replacement for sufficient DRAM.

Recommended Memory Improvements

A suitable memory hierarchy for the cloud server is:

CPU → L1 Cache → L2 Cache → Shared L3 Cache → High-Capacity DRAM → NVMe SSD

L1 and L2 caches should handle frequently accessed instructions and data for each processor core. A **large shared L3 cache** can store commonly accessed database data and improve sharing between multiple cores running parallel queries.

The server should use **high-capacity, high-bandwidth DRAM** to keep the active database working set in physical memory. This reduces page faults and prevents frequent swapping.

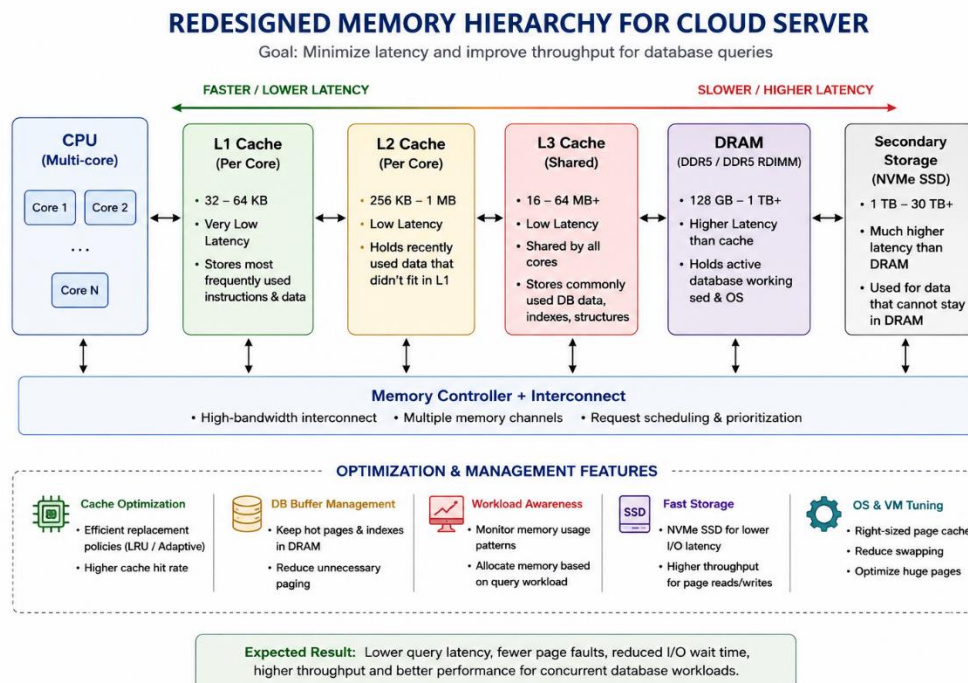
For secondary storage and unavoidable paging, **NVMe SSDs** should be used because they provide much lower latency and higher throughput than traditional HDD-based storage.

The database buffer manager can also prioritize frequently accessed pages so that important indexes and records remain in DRAM instead of repeatedly being loaded from storage.

Proposed Optimization

The redesigned system focuses on the following:

- Larger and efficient shared **L3 cache** for frequently accessed database data.
- Sufficient **high-capacity DRAM** to hold the active working set.
- Improved cache replacement policies to retain frequently used data.
- Reduced unnecessary paging and swapping.
- **NVMe SSD** as fast secondary storage for pages that cannot remain in DRAM.
- Memory allocation and database buffer tuning based on the workload.



Performance Analysis

In the original system, a cache miss followed by a DRAM shortage can result in page faults, forcing the system to retrieve data from secondary storage. This creates a large delay in database query execution.

With the proposed hierarchy, frequently accessed data is served from L1, L2 or L3 cache whenever possible. Larger DRAM capacity reduces page faults, while NVMe SSD reduces the penalty when storage access is unavoidable.

Therefore, the redesigned memory system can provide:

- Lower database query latency
- Reduced page faults
- Fewer accesses to slow secondary storage
- Better cache hit rate
- Higher throughput for concurrent queries
- More stable performance during peak workloads

Conclusion

For a cloud database server, improving cache performance alone is not enough. The system requires a balanced memory hierarchy consisting of efficient caches, sufficient high-speed DRAM and fast NVMe-based secondary storage. This reduces excessive paging, improves memory access speed and ultimately provides faster and more consistent database query performance.

3. Autonomous Vehicle Data Processing

An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

ANSWER :

Autonomous Vehicle Data Processing

An autonomous vehicle continuously receives data from cameras, LiDAR, radar, ultrasonic sensors and other vehicle systems. This data must be processed in real time to support functions such as object detection, lane detection and decision-making. Any delay in accessing sensor data can affect the response time of the vehicle. Therefore, the memory system should provide both **very low latency for critical operations** and **high bandwidth for handling large sensor data streams**.

Evaluation of SRAM, DRAM and MRAM

Memory Technology	Advantages	Limitations	Suitable Use
SRAM	Very low latency and fast access	Expensive and low capacity	Cache and critical real-time data
DRAM	High capacity and high bandwidth	Higher latency than SRAM and volatile	Sensor data, AI models and intermediate processing
MRAM	Non-volatile, fast and low standby power	Lower density and higher cost compared to DRAM	Critical data and fast recovery

SRAM is suitable for storing frequently accessed instructions and time-critical data because of its very low access latency. However, its high cost and low density make it unsuitable for storing large amounts of sensor data.

DRAM provides the capacity and bandwidth required for processing large data streams from cameras, LiDAR and radar. It is therefore the main working memory of the autonomous system.

MRAM is useful for storing important system states and critical data because it is non-volatile. Unlike DRAM, it retains data even when power is interrupted and can support faster system recovery.

Proposed Memory Hierarchy

The proposed memory hierarchy is:

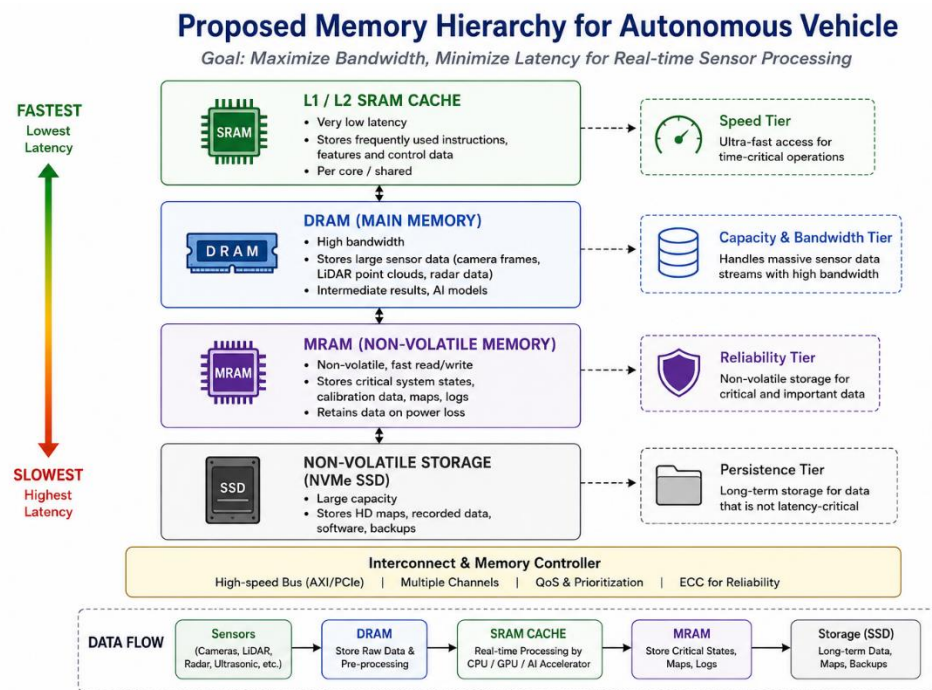
Processing Units / AI Accelerator → SRAM Cache → High-Bandwidth DRAM → MRAM → Non-volatile Storage

The hierarchy can be designed as follows:

- **SRAM:** Used as L1/L2 cache for frequently accessed sensor features, instructions and time-critical control data.
- **High-bandwidth DRAM:** Used as the main memory for large camera frames, LiDAR point clouds, radar data and intermediate AI processing results.

- **MRAM:** Used to store critical vehicle states, system configuration and important safety-related information that must be retained during power interruptions.
- A **high-speed memory controller** should prioritize safety-critical data and support parallel access to DRAM for multiple sensor-processing tasks.

To reduce unnecessary delays, sensor data can be processed in parallel, while frequently reused information remains in SRAM. Large data that cannot fit in SRAM is transferred to DRAM using high-bandwidth channels. MRAM acts as a reliable non-volatile layer for important information.



Performance Analysis

The proposed design separates data based on how quickly it is required and how much memory it needs. Time-critical information remains close to the processor in SRAM, reducing access latency. Large sensor datasets are stored in DRAM, which provides the bandwidth required for continuous real-time processing.

MRAM improves reliability by retaining critical information without continuous power. This reduces recovery time and provides an additional level of safety.

Therefore, the proposed hierarchy provides:

- Very low latency for critical real-time operations using SRAM
- High bandwidth for camera, LiDAR and radar data using DRAM
- Non-volatile storage of critical information using MRAM
- Reduced processor waiting time

- Better parallel processing of multiple sensor streams
- Improved response time and reliability

Conclusion

A single memory technology cannot meet all the requirements of an autonomous vehicle. SRAM provides the speed required for critical processing, DRAM provides the capacity and bandwidth needed for large sensor datasets, and MRAM improves reliability through non-volatility. By combining these technologies in a hierarchical structure, the autonomous vehicle can achieve lower latency, higher bandwidth and more reliable real-time data processing.

4. AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

ANSWER :

AI Inference Edge Device

An edge AI device performs AI inference locally instead of continuously depending on a cloud server. Examples include smart cameras, wearable devices and industrial monitoring systems. The main challenge is to load the AI model quickly while maintaining low power consumption. Since model files can be large, a single memory technology cannot provide the required combination of high capacity, low latency and energy efficiency.

Therefore, NAND Flash, DRAM and RRAM can be used at different levels of the memory hierarchy.

Comparison of NAND Flash, DRAM and RRAM

Memory Technology	Advantages	Limitations	Suitable Use
NAND Flash	Non-volatile, high capacity and low cost	Higher latency and slower than DRAM	Permanent storage of AI models
DRAM	High bandwidth and fast access	Volatile and consumes power	Temporary loading and execution of models
RRAM	Non-volatile, low power and fast access	Limited capacity and higher cost	Frequently used model parameters and AI weights

NAND Flash is suitable for storing the complete AI model permanently because it provides large storage capacity and retains data without power.

DRAM acts as the main working memory. When an AI model is required, the model or required sections are loaded from NAND Flash into DRAM for faster execution. However, DRAM continuously consumes power because data must be refreshed.

RRAM combines non-volatility with low power consumption. It can be used to store frequently accessed AI model weights or important parameters closer to the AI accelerator, reducing repeated loading from NAND Flash or DRAM.

Proposed Hierarchical Memory Structure

The proposed structure is:

AI Processor / NPU → RRAM → DRAM → NAND Flash

The memory hierarchy works as follows:

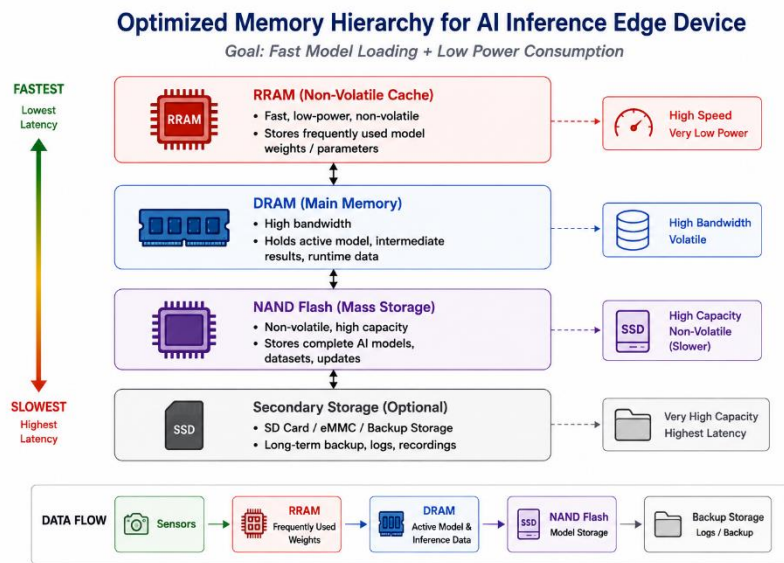
- **RRAM** is placed close to the AI processor and stores frequently used model weights and parameters for fast, low-power access.
- **DRAM** stores the active model, intermediate results and temporary data during inference.
- **NAND Flash** stores complete AI models and less frequently used data permanently.

When the device starts, the required model can be loaded from NAND Flash into DRAM. Frequently used weights can be retained in RRAM, reducing repeated transfers and improving inference speed.

Optimization Strategy

To improve performance and power efficiency:

- Store complete models in **NAND Flash**.
- Load only the required model layers or parameters into **DRAM** instead of loading unnecessary data.
- Retain frequently used weights in **RRAM**.
- Use intelligent prefetching to load the next required model data before it is requested.
- Power down unused DRAM sections whenever possible.
- Place RRAM close to the AI accelerator to reduce data movement.



Performance Analysis

In a conventional design, the entire model may need to be repeatedly transferred from NAND Flash to DRAM, increasing model loading time and energy consumption.

In the proposed hierarchy, NAND Flash provides high-capacity storage, DRAM provides the bandwidth required for active inference, and RRAM reduces repeated data transfers by retaining frequently used parameters even when power is removed.

This results in:

- Faster model loading
- Reduced memory access latency
- Lower power consumption
- Less data movement between memory levels
- Faster AI inference
- Better suitability for battery-powered edge devices

Conclusion

The optimized hierarchy combines the strengths of all three memory technologies. NAND Flash provides high-capacity non-volatile storage, DRAM provides high-speed working memory, and RRAM provides low-power, fast access to frequently used AI parameters. Thus, the structure **AI Processor** → **RRAM** → **DRAM** → **NAND Flash** can improve model loading speed while reducing overall power consumption.

5. High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

ANSWER :

High-Performance Gaming Console

A gaming console may experience frame drops during large scene loading because huge amounts of textures, geometry, shaders and game assets need to be accessed quickly. If frequently required data cannot fit in the **L3 cache**, the processor must fetch it from **DRAM**, which has much higher access latency. The question specifically focuses on analysing **L3 cache vs DRAM throughput** and proposing improvements to the memory hierarchy.

L3 Cache vs DRAM

L3 Cache	DRAM
Located closer to the processor	Main system memory
Lower latency	Higher latency
Smaller capacity	Much larger capacity
Useful for frequently reused game data	Stores large textures, models and scene assets
Reduces CPU waiting time	Provides high overall bandwidth for large data transfers

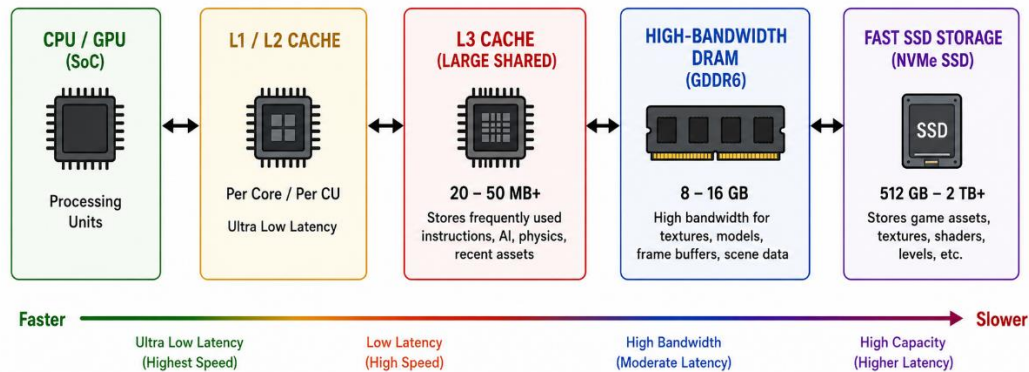
Proposed Memory Hierarchy Enhancement

CPU/GPU → L1/L2 Cache → Large Shared L3 Cache → High-Bandwidth DRAM → Fast SSD Storage

The proposed improvements are:

- Increase the **shared L3 cache capacity** to retain frequently reused game data.
- Use **high-bandwidth DRAM** to transfer large textures and scene data quickly.
- Use separate or optimized memory access paths for the **CPU and GPU** to reduce memory contention.
- Apply **prefetching**, where upcoming scene assets are loaded into cache or DRAM before they are actually required.
- Stream game assets gradually instead of loading the entire scene at once.
- Use a **fast SSD** for quick transfer of new game assets into DRAM.

ENHANCED MEMORY HIERARCHY FOR GAMING CONSOLE



How the Redesigned Hierarchy Reduces Frame Drops

When a new scene is about to load, required textures and assets can first be transferred from the SSD to DRAM. Frequently reused data is then retained in the L3 cache. During rendering, the CPU and GPU can access the required data from the faster cache whenever possible instead of repeatedly waiting for DRAM.

This reduces memory stalls and provides a more continuous flow of data to the processing units.

Expected Performance Improvement

The enhanced hierarchy can provide:

- Lower memory access latency
- Better utilisation of L3 cache
- Higher effective DRAM throughput
- Faster loading of large scenes and textures
- Reduced CPU/GPU memory stalls
- Fewer frame drops and smoother gameplay

Conclusion

L3 cache and DRAM serve different purposes in a gaming console. L3 cache provides low-latency access to frequently used data, while DRAM provides the capacity and bandwidth required for large game assets. A larger shared L3 cache, high-bandwidth DRAM, intelligent prefetching and fast asset streaming can together reduce frame drops during large scene loading.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Industry Problem (20%)	Application of Engineering Concepts (25%)	Solution Design (25%)	Use of Tools (15%)	Analysis (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools

Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
----------	-----	---	--	--	---------------------